

Algoritmos_entregable_Grupo4_EMM_MMT

July 29, 2026

Nombre y Apellidos: EMILIO FERNANDO MORENO MUÑOZ

GRUPO 4 CON MARCOS MILLA TRANCA

Url: <https://github.com/.../03MAIR—Algoritmos-de-Optimizacion—2019/tree/master/SEMINARIO>

Url: <https://github.com/Hiper1961/algoritmos.git>

Problema: 1. Sesiones de doblaje 2, Organizar los horarios de partidos de La Liga 3. Combinar cifras y operaciones

Descripcion del problema: 2. Desde la La Liga de fútbol profesional se pretende organizar los horarios de los partidos de liga de cada jornada. Se conocen algunos datos que nos deben llevar a diseñar un algoritmo que realice la asignación de los partidos a los horarios de forma que maximice la audiencia.

[]: 1. ¿Cuántas posibilidades hay sin tener en cuenta las restricciones?

- 10 partidos distintos en una jornada.
- 10 horarios disponibles en total (1 el viernes, 4 el sábado, 4 el domingo, 1 el lunes).

$$1+4+4+1 = 10$$

Sin restriccion:

$$10^{10} = 10,000,000,000$$

es decir: 10 mil millones de combinaciones posibles

```
[ ]: partidos = 10
horarios = 10

# Sin restricciones
total_sin_restricciones = horarios ** partidos

print(f"Posibilidades SIN restricciones: {total_sin_restricciones:,}")
```

2. ¿Cuántas posibilidades hay teniendo en cuenta todas las restricciones?

Resriccion: partido obligatorio lunes y viernes

tiene que existir obligatoriamente un partido el viernes y lunes. no pueden estar vacios

Bolsa Total:

$$10^{10} = 10,000,000,000$$

Viernes Vacío:

$$9^{10} = 3,486,784,401$$

Lunes Vacío:

$$9^{10} = 3,486,784,401$$

Corrigiendo el la inteseccion de los conjuntos que se resto doble:

Inteseccion donde ambos son vacios:

$$8^{10} = 1,073,741,824$$

Combinaciones Válidas =

$$Total - (ViernesVaco + LunesVaco) + AmbosVacos$$

$$CombinacionesVlidas = 10,000,000,000 - 6,973,568,802 + 1,073,741,824 = 4,100,173,022$$

```
[5]: sin_viernes = (horarios - 1) ** partidos
sin_lunes = (horarios - 1) ** partidos
sin_ambos = (horarios - 2) ** partidos

total_con_restricciones = total_sin_restricciones - (sin_viernes + sin_lunes) +
↪ sin_ambos

print(f"Posibilidades SIN restricciones: {total_sin_restricciones:,}")
print(f"Posibilidades CON restricciones (Al menos 1): {total_con_restricciones:
↪ ,}")
```

Posibilidades SIN restricciones: 10,000,000,000

Posibilidades CON restricciones (Al menos 1): 4,100,173,022

3. ¿Cuál es la estructura de datos que mejor se adapta al problema?

- Diccionarios para Tablas de datos referenciales: es preferible usar diccionarios como tabla de consulta en vez de ir iterando y consultando datos o calculando cada vez que se necesita. Con un diccionario solo se llama a la llave:valor
- Listas como acumuladores/ almacén de posible solución: esta estructura es ideal para recibir los datos de solución debido a que puedes mutar, poblar y eliminar valores directamente sobre el contenedor de datos

```
[2]: # O(1) de acceso usando Diccionarios para datos estáticos
audiencia_base = {
    'A-A': 2.0,
    'A-B': 1.3,
    'A-C': 1.0,
    'B-A': 1.3,
    'B-B': 0.9,
    'B-C': 0.75,
    'C-B': 0.75,
    'C-A': 1.0,
    'C-C': 0.47
}

ponderacion_horario = {
    'V20': 0.4,
    'S12': 0.55, 'S16': 0.7, 'S18': 0.8, 'S20': 1.0,
    'D12': 0.45, 'D16': 0.75, 'D18': 0.85, 'D20': 1.0,
    'L20': 0.4
}

# Cada índice: Partido
# el valor: llave del horario

# Diccionario
# llave: número de coincidencias
# valor: multiplicador restante (100% - castigo)
penalizacion_coincidencias = {
    0: 1.0,      # 0% castigo
    1: 0.75,     # 25% castigo
    2: 0.55,     # 45% castigo
    3: 0.40,     # 60% castigo
    4: 0.30,     # 70% castigo
    5: 0.25,     # 75% castigo
    6: 0.22,     # 78% castigo
    7: 0.20,     # 80% castigo
    8: 0.20,     # 80% castigo
    9: 0.20,     # 80% castigo
    10: 0.20     # 80% castigo
}

# Esta es la estructura dinámica que el algoritmo va a modificar constantemente.
```

```
estado_solucion_ejemplo = ['V20', 'S12', 'S16', 'S18', 'S20', 'D16', 'D16',  
↪ 'D18', 'D20', 'L20']
```

4. ¿Cuál es la función objetivo?

la función objetivo es la suma de las audiencias corregidas los 10 partidos y buscar ser maximizada

- La función objetivo Z representa la audiencia total calculada para toda la jornada de 10 partidos, esta misma es la que busca ser maximizada.
- Para cada partido p , la audiencia se obtiene multiplicando su audiencia base (la categoría de los equipos) por el coeficiente de franja horaria asignado y por el factor de corrección por coincidencia (si comparte horario con otros partidos)

$$Z = \sum_{p=1}^{10} \left(\text{Base}_p \times \text{Ponderación Horario}_p \times \text{Factor Coincidencia}_p \right)$$

el objetivo del algoritmo es encontrar el máximo valor de Z respetando la obligatoriedad de las restricciones de lunes y viernes

5. ¿Es un problema de maximización o minimización?

es claramente un problema de maximización, el enunciado principal del problema es encontrar una forma que maximice la audiencia en los partidos

5. Diseña un algoritmo para resolver el problema por Fuerza Bruta

-paso 1. el algoritmo consiste en generar absolutamente todas las combinaciones posibles de horarios para los 10 partidos

-paso 2. filtrar las que no cumplen la restricción (viernes y lunes obligatorios)

-paso 3. evaluar la función objetivo para cada combinación válida

-paso 4. quedarnos con la que dé el número mayor.

```
[ ]: import itertools  
import time  
  
# =====  
# FUNCIONES DEL ALGORITMO  
# =====  
  
def calcular_audiencia(combinacion, partidos):  
    """  
    Calcula la audiencia total de una asignación específica de horarios.  
    - combinacion: tupla con los horarios asignados, ej: ('V20', 'S12', 'S12', .  
    ↪ ..)  
    - partidos: lista de diccionarios con la info de cada partido.  
    """  
    audiencia_total = 0.0
```

```

    # Paso A: Contar cuántos partidos cayeron en cada horario para saber la
    ↪penalización
    conteo_horarios = {}
    for h in combinacion:
        if h in conteo_horarios:
            conteo_horarios[h] += 1
        else:
            conteo_horarios[h] = 1

    # Paso B: Calcular la audiencia partido por partido
    for i in range(len(partidos)):
        horario_asignado = combinacion[i]
        categoria_partido = partidos[i]['categoria']

        # 1. Audiencia Base
        base = audiencia_base[categoria_partido]

        # 2. Multiplicador por el horario
        pond = ponderacion_horario[horario_asignado]

        # 3. Factor de castigo por coincidencias
        cantidad_partidos_aqui = conteo_horarios[horario_asignado]
        factor_coincidencia = penalizacion_coincidencias[cantidad_partidos_aqui]

        # Multiplicación final para este partido
        audiencia_partido = base * pond * factor_coincidencia
        audiencia_total += audiencia_partido

    return audiencia_total

def resolver_fuerza_bruta(partidos_a_evaluar, horarios_disponibles):
    mejor_audiencia = 0.0
    mejor_combinacion = None
    iteraciones_validas = 0

    print(f"Generando permutaciones para {len(partidos_a_evaluar)} partidos en
    ↪{len(horarios_disponibles)} horarios...")

    # Genera el producto cartesiano: todas las formas de asignar horarios a los
    ↪partidos
    todas_las_combinaciones = itertools.product(horarios_disponibles,
    ↪repeat=len(partidos_a_evaluar))

    for combinacion in todas_las_combinaciones:
        # Filtro: OBLIGATORIO que exista al menos un partido el V20 y un
        ↪partido el L20

```

```

    # (Si estamos probando con un subset pequeño que no incluye V20 o L20,
    ↪saltamos este filtro)
    if ('V20' in horarios_disponibles and 'V20' not in combinacion) or \
        ('L20' in horarios_disponibles and 'L20' not in combinacion):
        continue

    iteraciones_validas += 1

    # Evaluar la función objetivo completa
    audiencia = calcular_audiencia(combinacion, partidos_a_evaluar)

    # Si encontramos una distribución que da más plata/audiencia, la
    ↪guardamos
    if audiencia > mejor_audiencia:
        mejor_audiencia = audiencia
        mejor_combinacion = combinacion

    return mejor_combinacion, mejor_audiencia, iteraciones_validas

# =====
# BLOQUE DE EJECUCIÓN Y PRUEBA (DATA DE INGRESO)
# =====

if __name__ == "__main__":
    # Ingresamos los 10 partidos exactos de la diapositiva ojo que esto es
    ↪hardcodeado. aquí no estoy usando ningun generador es tal cual la diapositiva
    jornada_completa = [
        {'id': 'Celta - Real Madrid', 'categoria': 'B-A'},
        {'id': 'Valencia - R. Sociedad', 'categoria': 'B-A'},
        {'id': 'Mallorca - Eibar', 'categoria': 'C-C'},
        {'id': 'Athletic - Barcelona', 'categoria': 'B-A'},
        {'id': 'Leganés - Osasuna', 'categoria': 'C-C'},
        {'id': 'Villarreal - Granada', 'categoria': 'B-C'},
        {'id': 'Alavés - Levante', 'categoria': 'B-B'},
        {'id': 'Espanyol - Sevilla', 'categoria': 'B-B'},
        {'id': 'Betis - Valladolid', 'categoria': 'B-C'},
        {'id': 'Atlético - Getafe', 'categoria': 'B-B'}
    ]

    lista_horarios_completa = ['V20', 'S12', 'S16', 'S18', 'S20', 'D12', 'D16',
    ↪'D18', 'D20', 'L20']

    # --- MODO DE PRUEBA RÁPIDA ---
    # Para que no se muera la pc vamos a evaluar solo los 4 primeros partidos
    ↪en 4 horarios.
    # Así verás el resultado en menos de 1 segundo.
    partidos_test = jornada_completa[:4]

```

```

horarios_test = ['V20', 'S20', 'D16', 'L20']

#horarios_test = lista_horarios_completa

print("=== INICIANDO ALGORITMO FUERZA BRUTA ===")

inicio = time.time()

# Llamamos a la función
combinacion_ganadora, max_audiencia, iteraciones = 
↪resolver_fuerza_bruta(partidos_test, horarios_test)

#guardamos el tiempo para poder calcular cuanto tarda despues
fin = time.time()

# --- RESULTADOS ESPERADOS (DATOS DE SALIDA) ---
print("\n=== RESULTADOS ===")
print(f"Tiempo de ejecución: {round(fin - inicio, 4)} segundos")
print(f"Combinaciones válidas evaluadas: {iteraciones}")
print(f"AUDIENCIA MÁXIMA ENCONTRADA: {round(max_audiencia, 2)} Millones\n")

print("Distribución óptima de horarios:")
for i in range(len(partidos_test)):
    print(f"-> {partidos_test[i]['id']} jugará el 
↪{combinacion_ganadora[i]}")

```

=== INICIANDO ALGORITMO FUERZA BRUTA ===
Generando permutaciones para 4 partidos en 4 horarios...

=== RESULTADOS ===
Tiempo de ejecución: 0.0004 segundos
Combinaciones válidas evaluadas: 110
AUDIENCIA MÁXIMA ENCONTRADA: 4.5 Millones

Distribución óptima de horarios:
-> Celta - Real Madrid jugará el V20
-> Valencia - R. Sociedad jugará el S20
-> Mallorca - Eibar jugará el L20
-> Athletic - Barcelona jugará el D16

6. Calcula la complejidad del algoritmo por Fuerza Bruta

Complejidad Temporal:

$O(H^P)$, donde H es el número de horarios disponibles (10) y P es el número de partidos (10). Estamos ante una complejidad exponencial. evaluar 10^{10} iteraciones en Python puro puede tomar horas o días dependiendo del procesador, en un lenguaje orientado a ejecutar en ciclos de reloj pueda que sea un poco mas eficiente, sin embargo sigue siendo alto. Es matemáticamente exacto,

pero computacionalmente inviable para problemas grandes.

Complejidad Espacial:

$O(P)$, ya que en memoria solo necesitamos guardar la combinación actual que se está evaluando (tamaño 10) y la mejor combinación encontrada hasta el momento. Si en lugar de iterar guardaras todo en una lista antes de procesarlo, la memoria explotaría a $O(H^P)$, lo cual sería un error garrafal. este tipo de algoritmo en coste de memoria ram suele ser muy eficiente por mas que el lenguaje no sea aplicado directamente sobre los registros de la memoria ram.

7. Diseña un algoritmo que mejore la complejidad

se plantea el algoritmo voraz El algoritmo Voraz no explora el universo completo de permutaciones como lo hace la fuerza bruta. En su lugar, construye una solución óptima **paso a paso**, tomando en cada iteración la mejor decisión local inmediata de forma irreversible.

Para maximizar la audiencia total de la jornada, sigue estas **3 reglas**:

- **Ordenamiento y Priorización:** Se ordenan los partidos de mayor a menor atractivo (basándose en la audiencia base). Se le da prioridad a la asignación de los partidos de mayor impacto.
- **Sacrificio Estratégico de Penalización:** La liga da horarios con alta penalización (Viernes 20h y Lunes 20h, factor de ponderación 0.4). El algoritmo asigna directamente a estos horarios los dos partidos de menor audiencia base para blindar la rentabilidad de los partidos estelares.
- **Asignación Local Óptima:** Para el resto de partidos (de mayor a menor valor), el algoritmo evalúa de forma iterativa los horarios disponibles restantes y fija el partido en el casillero que entregue la máxima audiencia instantánea.

```
[3]: import time

# =====
# FUNCIONES DEL ALGORITMO VORAZ
# =====

def calcular_audiencia_parcial(asignaciones_actuales):
    """
    Calcula la audiencia de la jornada con los partidos asignados hasta el
    momento.
    """
    conteo_horarios = {}
    for asig in asignaciones_actuales:
        h = asig['horario']
        conteo_horarios[h] = conteo_horarios.get(h, 0) + 1

    audiencia_total = 0.0
    for asig in asignaciones_actuales:
        cat = asig['partido']['categoria']
        h = asig['horario']
```



```

        base = audiencia_base[cat]
        pond = ponderacion_horario[h]
        factor_coincidencia = penalizacion_coincidencias[conteo_horarios[h]]

        audiencia_total += base * pond * factor_coincidencia

    return audiencia_total

def resolver_greedy(partidos_lista, horarios):
    # 1. ORDENAR DE MAYOR A MENOR AUDIENCIA BASE
    # Así nos aseguramos de posicionar primero los mejores partidos
    partidos_ordenados = sorted(partidos_lista, key=lambda x:
    ↪audiencia_base[x['categoria']], reverse=True)

    asignaciones_finales = []

    # 2. CUMPLIR RESTRICCIONES INTELIGENTEMENTE
    # Extraemos los 2 peores partidos (los últimos de la lista) para
    ↪sacrificarlos el Viernes y Lunes
    partido_para_viernes = partidos_ordenados.pop(-1)
    partido_para_lunes = partidos_ordenados.pop(-1)

    asignaciones_finales.append({'partido': partido_para_viernes, 'horario':
    ↪'V20'})
    asignaciones_finales.append({'partido': partido_para_lunes, 'horario':
    ↪'L20'})

    # 3. ALGORITMO VORAZ PARA EL RESTO DE PARTIDOS
    # Iteramos los partidos fuertes que quedan
    for partido in partidos_ordenados:
        mejor_horario = None
        mejor_audiencia_simulada = -1

        # Probamos este partido en cada uno de los 10 horarios disponibles
        for h in horarios:
            asignacion_simulada = asignaciones_finales.copy()
            asignacion_simulada.append({'partido': partido, 'horario': h})

            # Verificamos cuánta audiencia total tendríamos si lo metemos aquí
            audiencia_simulada = calcular_audiencia_parcial(asignacion_simulada)

            # Si mejora lo que hemos visto, guardamos este horario como el
            ↪mejor candidato
            if audiencia_simulada > mejor_audiencia_simulada:
                mejor_audiencia_simulada = audiencia_simulada
                mejor_horario = h

```

```

        # Una vez probados todos los horarios, lo asignamos definitivamente al
        ↪ que dio más plata/audiencia
        asignaciones_finales.append({'partido': partido, 'horario':
        ↪ mejor_horario})

    # Calculamos el resultado final
    audiencia_maxima = calcular_audiencia_parcial(asignaciones_finales)

    return asignaciones_finales, audiencia_maxima

# =====
# BLOQUE DE EJECUCIÓN (JORNADA COMPLETA)
# =====

if __name__ == "__main__":
    jornada_completa = [
        {'id': 'Celta - Real Madrid', 'categoria': 'B-A'},
        {'id': 'Valencia - R. Sociedad', 'categoria': 'B-A'},
        {'id': 'Mallorca - Eibar', 'categoria': 'C-C'},
        {'id': 'Athletic - Barcelona', 'categoria': 'B-A'},
        {'id': 'Leganés - Osasuna', 'categoria': 'C-C'},
        {'id': 'Villarreal - Granada', 'categoria': 'B-C'},
        {'id': 'Alavés - Levante', 'categoria': 'B-B'},
        {'id': 'Espanyol - Sevilla', 'categoria': 'B-B'},
        {'id': 'Betis - Valladolid', 'categoria': 'B-C'},
        {'id': 'Atlético - Getafe', 'categoria': 'B-B'}
    ]

    lista_horarios_completa = ['V20', 'S12', 'S16', 'S18', 'S20', 'D12', 'D16',
    ↪ 'D18', 'D20', 'L20']

    print("=== INICIANDO ALGORITMO VORAZ (GREEDY) ===")
    inicio = time.time()

    # Procesamos los 10 partidos con los 10 horarios de un golpe
    asignaciones, max_audiencia = resolver_greedy(jornada_completa,
    ↪ lista_horarios_completa)

    fin = time.time()

    print("\n=== RESULTADOS ===")
    print(f"Tiempo de ejecución: {round(fin - inicio, 6)} segundos")
    print(f"AUDIENCIA MÁXIMA ALCANZADA: {round(max_audiencia, 2)} Millones\n")

    print("Distribución óptima de horarios (Heurística):")
    # Para imprimirlo bonito, ordenamos por día
    orden_dias = {h: i for i, h in enumerate(lista_horarios_completa)}

```

```

asignaciones.sort(key=lambda x: orden_dias[x['horario']])

for item in asignaciones:
    print(f"-> Horario {item['horario']}: {item['partido']['id']} (Cat:␣
↪{item['partido']['categoria']})")

```

=== INICIANDO ALGORITMO VORAZ (GREEDY) ===

=== RESULTADOS ===

Tiempo de ejecución: 0.000217 segundos

AUDIENCIA MÁXIMA ALCANZADA: 5.14 Millones

Distribución óptima de horarios (Heurística):

```

-> Horario V20: Leganés - Osasuna (Cat: C-C)
-> Horario S12: Villarreal - Granada (Cat: B-C)
-> Horario S16: Atlético - Getafe (Cat: B-B)
-> Horario S18: Alavés - Levante (Cat: B-B)
-> Horario S20: Celta - Real Madrid (Cat: B-A)
-> Horario D12: Betis - Valladolid (Cat: B-C)
-> Horario D16: Espanyol - Sevilla (Cat: B-B)
-> Horario D18: Athletic - Barcelona (Cat: B-A)
-> Horario D20: Valencia - R. Sociedad (Cat: B-A)
-> Horario L20: Mallorca - Eibar (Cat: C-C)

```

8. Calcula la complejidad del algoritmo mejorado

El análisis del tiempo de ejecución del **Algoritmo Voraz** se desglosa según las estructuras de control empleadas:

1. **Ordenamiento de entrada:** $\mathcal{O}(P \log P)$ usando Timsort sobre los P partidos.
2. **Asignación de restricciones fijas:** $\mathcal{O}(1)$ para el aislamiento de los dos partidos de menor rendimiento.
3. **Exploración Greedy:** Un bucle anidado que recorre los $P - 2$ partidos restantes contra los H horarios disponibles, resultando en $\mathcal{O}(P \cdot H)$ evaluaciones numéricas.

Complejidad Temporal Total:

$$\mathcal{O}(P \log P + P \cdot H)$$

Calculando la temporal:

en este caso para $P = 10$ y $H = 10$

el ordenamiento:

$$10 \cdot \log_2(10) = 10 \cdot 3.3219 \approx \mathbf{33.2} \text{ operaciones}$$

los 2 partidos mandados directos:

2 operaciones

el bucle anidado: evaluamos los 8 partidos restantes en los 10 horarios

$$8 \times 10 = \mathbf{80} \text{ evaluaciones}$$

hasta el ultimo partido:

$$\text{Total Exacto} = 33 + 2 + 80 = \mathbf{115} \text{ operaciones}$$

Conclusión del Cálculo: Para la jornada estándar de $P = 10$ y $H = 10$, el algoritmo Voraz reduce el volumen de procesamiento de 10^{10} **operaciones (Fuerza Bruta)** a aproximadamente 115 **operaciones efectivas**. Esto representa una reducción del 99.999999885% **en la carga computacional del procesador**, manteniendo una respuesta prácticamente instantánea (~ 0.0001 segundos).

Complejidad Espacial Total:

$$\mathcal{O}(P + H)$$

La complejidad espacial del **Algoritmo Voraz** es estrictamente **lineal**:

Justificación: 1. **Estructuras de datos:** Solo requiere almacenar en RAM la lista de P partidos, el conjunto de H horarios y el diccionario final de asignaciones de tamaño P .

2. **Uso de Pila (Stack):** Al ser un enfoque estrictamente iterativo (sin recursión), no genera sobrecarga en la pila de llamadas de la CPU ($\mathcal{O}(1)$).
3. **Eficiencia en RAM:** Para el caso $P = 10$ y $H = 10$, el consumo de memoria es de apenas unos bytes (despreciable considerando la cantidad enorme que tiene mi pc 32gb), evitando el riesgo de desbordamiento de memoria (**MemoryError**) que ocurre al intentar almacenar matrices combinatorias masivas en la Fuerza Bruta.

```
[17]: a= 10**10
      b=87

      print(100-(b/a), "%")
```

99.9999999913 %

9. Diseña un juego de datos de entrada aleatorio

```
[1]: import random

def generar_jornada_aleatoria():
    # 1. Clasificación lógica de equipos por nivel/tier
    equipos_tier = {
        # Tier A (Top mundial / pelean la liga)
        'Real Madrid': 'A', 'Barcelona': 'A', 'Atlético Madrid': 'A',

        # Tier B (Pelean Europa / mitad superior)
        'Sevilla': 'B', 'Real Sociedad': 'B', 'Athletic Club': 'B',
```

```

    'Villarreal': 'B', 'Betis': 'B', 'Valencia': 'B',

    # Tier C (Luchan permanencia / mitad inferior)
    'Celta': 'C', 'Mallorca': 'C', 'Osasuna': 'C', 'Granada': 'C',
    'Alavés': 'C', 'Espanyol': 'C', 'Getafe': 'C', 'Leganés': 'C',
    'Eibar': 'C', 'Valladolid': 'C', 'Rayo Vallecano': 'C'
}

# 2. Seleccionamos 20 equipos al azar sin repetir
equipos = list(equipos_tier.keys())
random.shuffle(equipos)

jornada_completa = []

# 3. Emparejamos de 2 en 2 para formar los 10 partidos
for i in range(0, 20, 2):
    eq1 = equipos[i]
    eq2 = equipos[i+1]

    tier1 = equipos_tier[eq1]
    tier2 = equipos_tier[eq2]

    # Ordenamos los tiers alfabéticamente para mantener consistencia
    # (Ej: siempre 'A-C' y nunca 'C-A')
    tiers_ordenados = sorted([tier1, tier2])
    categoria = f"{tiers_ordenados[0]}-{tiers_ordenados[1]}"

    # 4. Guardamos en el formato exacto de lista de diccionarios
    jornada_completa.append({
        'id': f"{eq1} - {eq2}",
        'categoria': categoria
    })

return jornada_completa

# --- PRUEBA DE EJECUCIÓN ---
jornada_test = generar_jornada_aleatoria()

# Imprime la lista tal cual el formato que pediste
import pprint
pprint.pprint(jornada_test)

```

```

[{'categoria': 'C-C', 'id': 'Leganés - Granada'},
 {'categoria': 'A-C', 'id': 'Rayo Vallecano - Barcelona'},
 {'categoria': 'B-C', 'id': 'Espanyol - Betis'},
 {'categoria': 'B-B', 'id': 'Real Sociedad - Sevilla'},
 {'categoria': 'C-C', 'id': 'Eibar - Celta'},

```

```
{'categoria': 'A-C', 'id': 'Atlético Madrid - Getafe'},
{'categoria': 'B-C', 'id': 'Athletic Club - Valladolid'},
{'categoria': 'C-C', 'id': 'Alavés - Mallorca'},
{'categoria': 'A-C', 'id': 'Real Madrid - Osasuna'},
{'categoria': 'B-B', 'id': 'Villarreal - Valencia']}
```

Probando en fuerza bruta

```
[30]: import itertools
import time

# =====
# FUNCIONES DEL ALGORITMO
# =====

def calcular_audiencia(combinacion, partidos):
    """
    Calcula la audiencia total de una asignación específica de horarios.
    - combinacion: tupla con los horarios asignados, ej: ('V20', 'S12', 'S12', .
    ↪...)
    - partidos: lista de diccionarios con la info de cada partido.
    """
    audiencia_total = 0.0

    # Paso A: Contar cuántos partidos cayeron en cada horario para saber la
    ↪penalización
    conteo_horarios = {}
    for h in combinacion:
        if h in conteo_horarios:
            conteo_horarios[h] += 1
        else:
            conteo_horarios[h] = 1

    # Paso B: Calcular la audiencia partido por partido
    for i in range(len(partidos)):
        horario_asignado = combinacion[i]
        categoria_partido = partidos[i]['categoria']

        # 1. Audiencia Base
        base = audiencia_base[categoria_partido]

        # 2. Multiplicador por el horario
        pond = ponderacion_horario[horario_asignado]

        # 3. Factor de castigo por coincidencias
        cantidad_partidos_aqui = conteo_horarios[horario_asignado]
        factor_coincidencia = penalizacion_coincidencias[cantidad_partidos_aqui]
```

```

        # Multiplicación final para este partido
        audiencia_partido = base * pond * factor_coincidencia
        audiencia_total += audiencia_partido

    return audiencia_total

def resolver_fuerza_bruta(partidos_a_evaluar, horarios_disponibles):
    mejor_audiencia = 0.0
    mejor_combinacion = None
    iteraciones_validas = 0

    print(f"Generando permutaciones para {len(partidos_a_evaluar)} partidos en {len(horarios_disponibles)} horarios...")

    # Genera el producto cartesiano: todas las formas de asignar horarios a los partidos
    todas_las_combinaciones = itertools.product(horarios_disponibles, repeat=len(partidos_a_evaluar))

    for combinacion in todas_las_combinaciones:
        # Filtro: OBLIGATORIO que exista al menos un partido el V20 y un partido el L20
        # (Si estamos probando con un subset pequeño que no incluye V20 o L20, saltamos este filtro)
        if ('V20' in horarios_disponibles and 'V20' not in combinacion) or \
            ('L20' in horarios_disponibles and 'L20' not in combinacion):
            continue

        iteraciones_validas += 1

        # Evaluar la función objetivo completa
        audiencia = calcular_audiencia(combinacion, partidos_a_evaluar)

        # Si encontramos una distribución que da más plata/audiencia, la guardamos
        if audiencia > mejor_audiencia:
            mejor_audiencia = audiencia
            mejor_combinacion = combinacion

    return mejor_combinacion, mejor_audiencia, iteraciones_validas

# =====
# 3. BLOQUE DE EJECUCIÓN Y PRUEBA (DATA DE INGRESO)
# =====

```

```

if __name__ == "__main__":

    jornada_completa = generar_jornada_aleatoria()

    lista_horarios_completa = ['V20', 'S12', 'S16', 'S18', 'S20', 'D12', 'D16',
↪ 'D18', 'D20', 'L20']

    # --- MODO DE PRUEBA RÁPIDA ---
    # Para que no se muera la pc vamos a evaluar solo los 4 primeros partidos
↪ en 4 horarios.
    # Así verás el resultado en menos de 1 segundo.
    partidos_test = jornada_completa[:4]
    horarios_test = ['V20', 'S20', 'D16', 'L20']

    #horarios_test = lista_horarios_completa

    print("=== INICIANDO ALGORITMO FUERZA BRUTA ===")

    inicio = time.time()

    # Llamamos a la función
    combinacion_ganadora, max_audiencia, iteraciones =
↪ resolver_fuerza_bruta(partidos_test, horarios_test)

    #guardamos el tiempo para poder calcular cuanto tarda despues
    fin = time.time()

    # --- RESULTADOS ESPERADOS (DATOS DE SALIDA) ---
    print("\n=== RESULTADOS ===")
    print(f"Tiempo de ejecución: {round(fin - inicio, 4)} segundos")
    print(f"Combinaciones válidas evaluadas: {iteraciones}")
    print(f"AUDIENCIA MÁXIMA ENCONTRADA: {round(max_audiencia, 2)} Millones\n")

    print("Distribución óptima de horarios:")
    for i in range(len(partidos_test)):
        print(f"-> {partidos_test[i]['id']} jugará el
↪ {combinacion_ganadora[i]}")

```

=== INICIANDO ALGORITMO FUERZA BRUTA ===

Generando permutaciones para 4 partidos en 4 horarios...

=== RESULTADOS ===

Tiempo de ejecución: 0.0001 segundos

Combinaciones válidas evaluadas: 110

AUDIENCIA MÁXIMA ENCONTRADA: 1.9 Millones

Distribución óptima de horarios:

-> Atlético Madrid - Alavés jugará el D16

-> Valladolid - Eibar jugará el V20

-> Betis - Real Madrid jugará el S20

-> Getafe - Athletic Club jugará el L20

Probando en voraz

```
[ ]: import time

# =====
# FUNCIONES DEL ALGORITMO VORAZ
# =====

def calcular_audiencia_parcial(asignaciones_actuales):
    """
    Calcula la audiencia de la jornada con los partidos asignados hasta el
    momento.
    """
    conteo_horarios = {}
    for asig in asignaciones_actuales:
        h = asig['horario']
        conteo_horarios[h] = conteo_horarios.get(h, 0) + 1

    audiencia_total = 0.0
    for asig in asignaciones_actuales:
        cat = asig['partido']['categoria']
        h = asig['horario']

        base = audiencia_base[cat]
        pond = ponderacion_horario[h]
        factor_coincidencia = penalizacion_coincidencias[conteo_horarios[h]]

        audiencia_total += base * pond * factor_coincidencia

    return audiencia_total

def resolver_greedy(partidos_lista, horarios):
    # 1. ORDENAR DE MAYOR A MENOR AUDIENCIA BASE
    # Así nos aseguramos de posicionar primero los mejores partidos
    partidos_ordenados = sorted(partidos_lista, key=lambda x:
    audiencia_base[x['categoria']], reverse=True)

    asignaciones_finales = []

    # 2. CUMPLIR RESTRICCIONES INTELIGENTEMENTE
```

```

    # Extraemos los 2 peores partidos (los últimos de la lista) para
    ↪sacrificarlos el Viernes y Lunes
    partido_para_viernes = partidos_ordenados.pop(-1)
    partido_para_lunes = partidos_ordenados.pop(-1)

    asignaciones_finales.append({'partido': partido_para_viernes, 'horario':
    ↪'V20'})
    asignaciones_finales.append({'partido': partido_para_lunes, 'horario':
    ↪'L20'})

    # 3. ALGORITMO VORAZ PARA EL RESTO DE PARTIDOS
    # Iteramos los partidos fuertes que quedan
    for partido in partidos_ordenados:
        mejor_horario = None
        mejor_audiencia_simulada = -1

        # Probamos este partido en cada uno de los 10 horarios disponibles
        for h in horarios:
            asignacion_simulada = asignaciones_finales.copy()
            asignacion_simulada.append({'partido': partido, 'horario': h})

            # Verificamos cuánta audiencia total tendríamos si lo metemos aquí
            audiencia_simulada = calcular_audiencia_parcial(asignacion_simulada)

            # Si mejora lo que hemos visto, guardamos este horario como el
            ↪mejor candidato
            if audiencia_simulada > mejor_audiencia_simulada:
                mejor_audiencia_simulada = audiencia_simulada
                mejor_horario = h

        # Una vez probados todos los horarios, lo asignamos definitivamente al
        ↪que dio más plata/audiencia
        asignaciones_finales.append({'partido': partido, 'horario':
        ↪mejor_horario})

    # Calculamos el resultado final
    audiencia_maxima = calcular_audiencia_parcial(asignaciones_finales)

    return asignaciones_finales, audiencia_maxima

# =====
# 3. BLOQUE DE EJECUCIÓN (JORNADA COMPLETA)
# =====

if __name__ == "__main__":
    jornada_completa = generar_jornada_aleatoria()

```

```

    lista_horarios_completa = ['V20', 'S12', 'S16', 'S18', 'S20', 'D12', 'D16',
↪ 'D18', 'D20', 'L20']

    print("=== INICIANDO ALGORITMO VORAZ (GREEDY) ===")
    inicio = time.time()

    # Procesamos los 10 partidos con los 10 horarios de un golpe
    asignaciones, max_audiencia = resolver_greedy(jornada_completa,
↪ lista_horarios_completa)

    fin = time.time()

    print("\n=== RESULTADOS ===")
    print(f"Tiempo de ejecución: {round(fin - inicio, 6)} segundos")
    print(f"AUDIENCIA MÁXIMA ALCANZADA: {round(max_audiencia, 2)} Millones\n")

    print("Distribución óptima de horarios (Heurística):")
    # Para imprimirlo bonito, ordenamos por día
    orden_dias = {h: i for i, h in enumerate(lista_horarios_completa)}
    asignaciones.sort(key=lambda x: orden_dias[x['horario']])

    for item in asignaciones:
        print(f"-> Horario {item['horario']}: {item['partido']['id']} (Cat:
↪ {item['partido']['categoria']})")

```

[]: Referencias:

-Brassard, G., y Bratley, P. (1997). Fundamentos de algoritmia. ISBN 13:
↪ 9788489660007

-Guerequeta, R., y Vallecillo, A. (2000). Técnicas de diseño de algoritmos.
<http://www.lcc.uma.es/~av/Libro/indice.html>

Describe brevemente en unas líneas como crees que es posible avanzar en el estudio del problema. Ten en cuenta incluso posibles variaciones del problema y/o variaciones al alza del tamaño: En mi opinion creo que incluso posibles variaciones del problema y/o variaciones en el incremento de variables teniendo una base como la mostrada en el ´presente trabajo nos permite tener posibilidades de solucion mas eficientes con los algoritmos de fuerza bruta y/o voraz. Siendo el algoritmo voraz el mas eficiente en tiempo de ejecucion, y con mayor audiencia maximizacion de la funcion con el de fuerza bruta.

[]: